

# FRAUD DETECTION ANALYSIS APP

## 1. Machine Learning Model Development and Deployment

A Machine Learning fraud detection model was developed to identify fraudulent financial transactions based on transaction-related features. The project also included the development of a Streamlit-based web application used to deploy the trained model and enable real-time fraud prediction through an interactive user interface.

## 2. Dataset Summary

The dataset consisted of approximately 6,362,620 records and 11 variables containing transaction-related information. The features included transaction type, transaction amount, sender and receiver account balances before and after transactions, and a target variable indicating whether a transaction was fraudulent or legitimate.

## 3. Data Quality Assessment

A data quality assessment was conducted to evaluate the completeness and reliability of the dataset. The analysis confirmed that the dataset contained no missing values, and the records were appropriately structured and formatted for machine learning analysis.

## 4. Exploratory Data Analysis (EDA) Using Python

Exploratory Data Analysis was performed using Python libraries such as Pandas to understand the structure and characteristics of the dataset.

The following steps were completed during the analysis:

- Imported the dataset using the Pandas library
- Reviewed the structure of the dataset using `df.head()`
- Examined data types, variable information, and dataset structure using `df.info()`
- Explored transaction-related patterns and feature distributions to support model development and data preprocessing processes.

```
df.head()
```

|   | step | type     | amount   | nameOrig    | oldbalanceOrg | newbalanceOrig | nameDest    | oldbalanceDest | newbalanceDest | isFraud | isFlaggedFraud |
|---|------|----------|----------|-------------|---------------|----------------|-------------|----------------|----------------|---------|----------------|
| 0 | 1    | PAYMENT  | 9839.64  | C1231006815 | 170136.0      | 160296.36      | M1979787155 | 0.0            | 0.0            | 0       | 0              |
| 1 | 1    | PAYMENT  | 1864.28  | C1666544295 | 21249.0       | 19384.72       | M2044282225 | 0.0            | 0.0            | 0       | 0              |
| 2 | 1    | TRANSFER | 181.00   | C1305486145 | 181.0         | 0.00           | C553264065  | 0.0            | 0.0            | 1       | 0              |
| 3 | 1    | CASH_OUT | 181.00   | C840083671  | 181.0         | 0.00           | C38997010   | 21182.0        | 0.0            | 1       | 0              |
| 4 | 1    | PAYMENT  | 11668.14 | C2048537720 | 41554.0       | 29885.86       | M1230701703 | 0.0            | 0.0            | 0       | 0              |

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6362620 entries, 0 to 6362619
Data columns (total 11 columns):
 #   Column                Dtype
---  -
 0   step                  int64
 1   type                  object
 2   amount                float64
 3   nameOrig              object
 4   oldbalanceOrg         float64
 5   newbalanceOrig        float64
 6   nameDest              object
 7   oldbalanceDest        float64
 8   newbalanceDest        float64
 9   isFraud               int64
10  isFlaggedFraud         int64
dtypes: float64(5), int64(3), object(3)
memory usage: 534.0+ MB
```

## 4. Data Analysis

- Reviewed the columns in the dataset to understand the available features and their relevance to fraud detection.
- Examined transaction-related variables such as transaction type, transaction amount, sender balances, and receiver balances.
- Identified categorical and numerical variables to support data preprocessing and machine learning model development.

```
: df.columns
```

```
: Index(['step', 'type', 'amount', 'nameOrig', 'oldbalanceOrg', 'newbalanceOrig',
        'nameDest', 'oldbalanceDest', 'newbalanceDest', 'isFraud',
        'isFlaggedFraud'],
        dtype='object')
```

- Counted the number of fraudulent transactions and transactions flagged as fraud to understand the distribution of fraud cases within the dataset.

```
df['isFraud'].value_counts()
```

```
isFraud
0      6354407
1         8213
Name: count, dtype: int64
```

```
df['isFlaggedFraud'].value_counts()
```

```
isFlaggedFraud
0      6362604
1          16
Name: count, dtype: int64
```

- Checked the dataset for null or missing values to ensure data completeness and quality.
- Reviewed the shape of the dataset to determine the total number of rows and columns available for analysis and model development.

```
df.isnull().sum().sum()
```

```
0
```

```
df.shape
#Result means (Rows, Columns)
```

```
(6362620, 11)
```

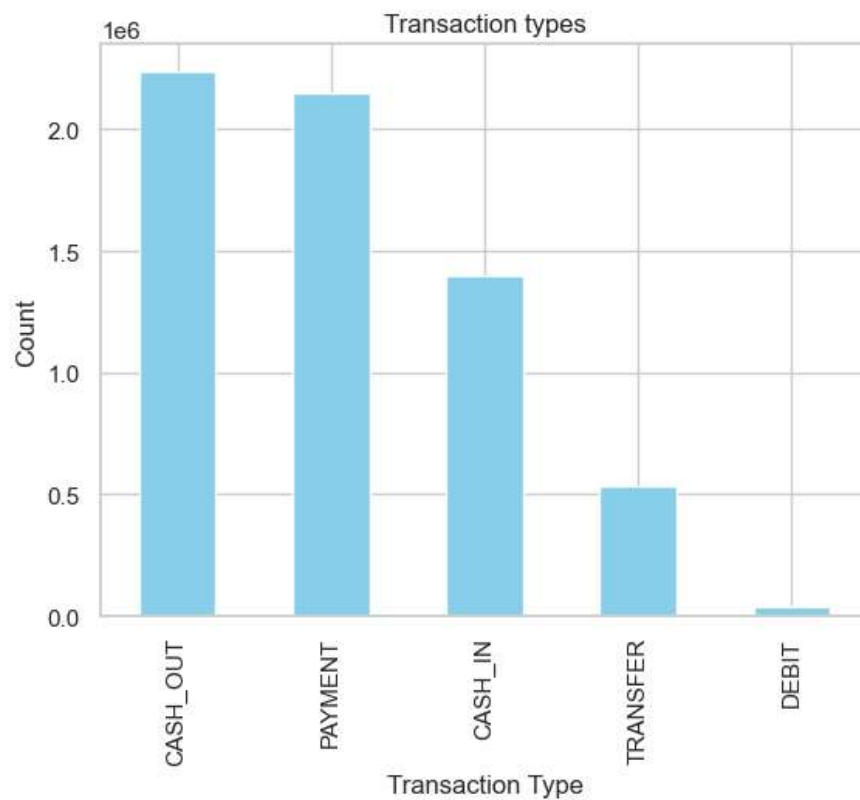
- Calculated the proportion of fraudulent transactions within the dataset and found that fraudulent transactions accounted for approximately 13% of the total records.

```
round((df['isFraud'].value_counts()[1]/df.shape[0]) * 100,2)
```

0.13

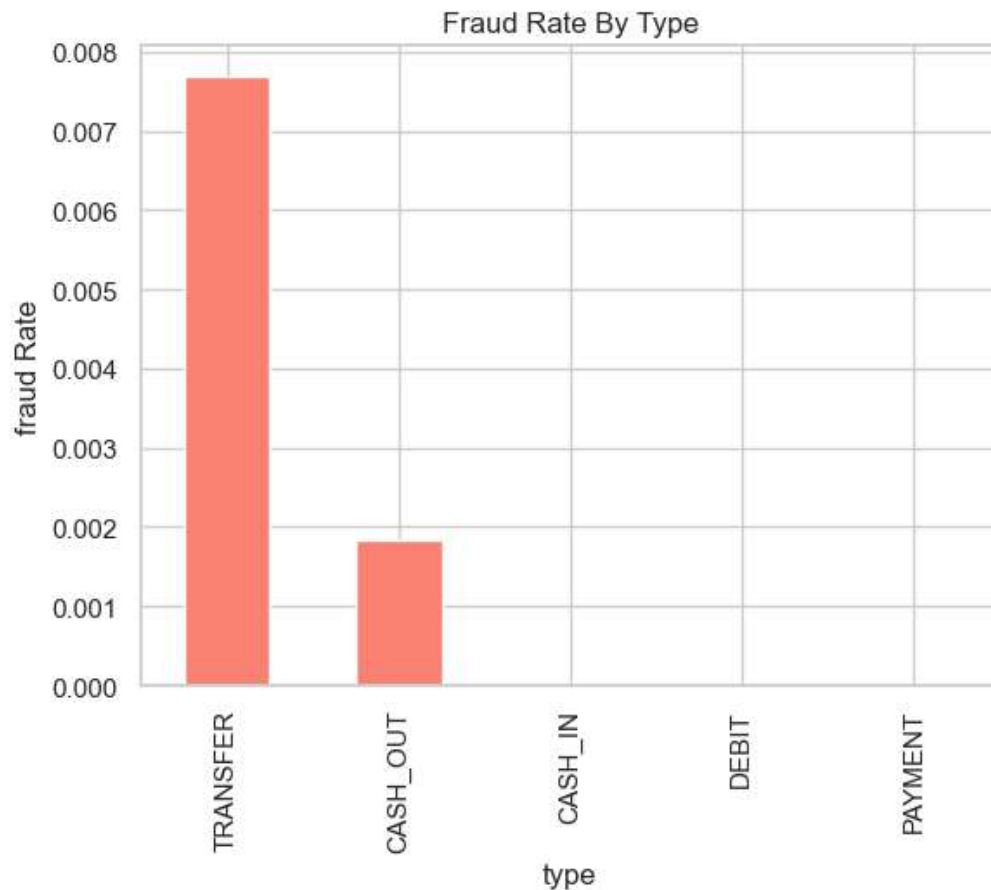
- Visualized the different transaction types to understand the distribution and frequency of payment activities within the dataset.

```
#Visualizing The transaction types
df['type'].value_counts().plot(kind = 'bar', title = 'Transaction types', color = 'skyblue')
plt.xlabel('Transaction Type')
plt.ylabel('Count')
plt.show()
```



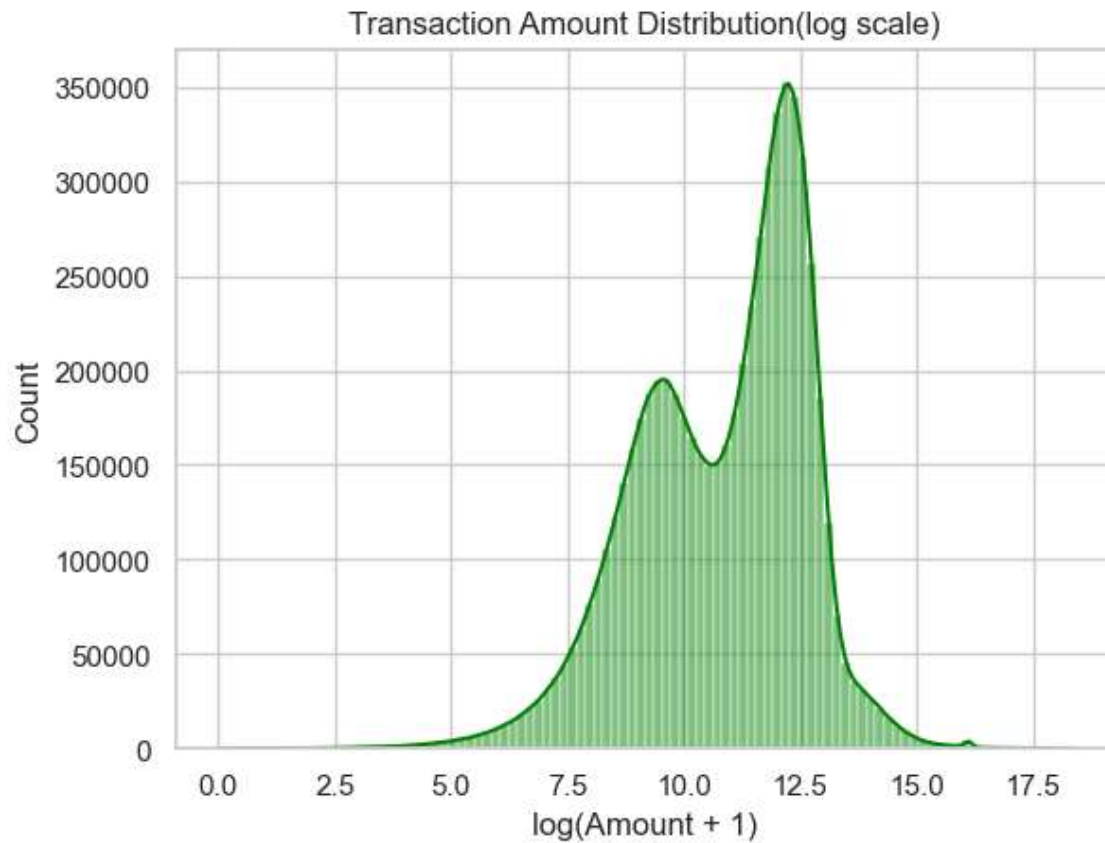
- Visualized the distribution of fraudulent transactions across different transaction types to identify which transaction categories were most associated with fraud.

```
fraud_by_type = df.groupby('type')['isFraud'].mean().sort_values(ascending=False)
fraud_by_type.plot(kind = 'bar', title = 'Fraud Rate By Type', color = 'salmon')
plt.ylabel('fraud Rate')
plt.show()
```



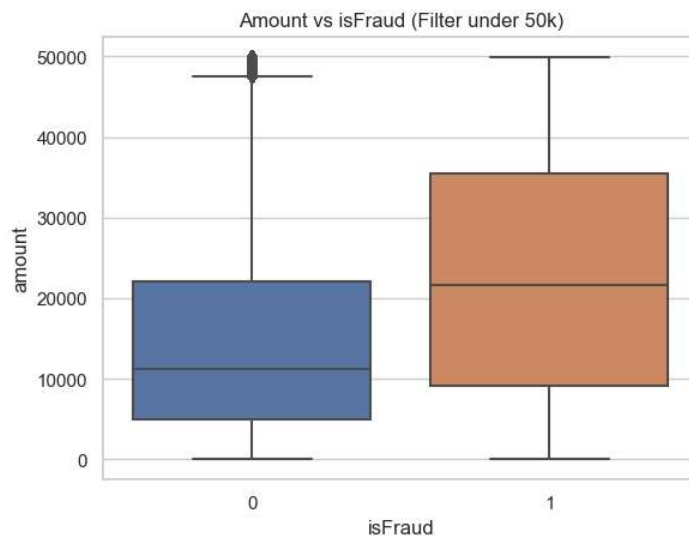
- Created a transaction distribution histogram to visualize the spread and frequency of transaction amounts within the dataset.

```
sns.histplot(np.log1p(df['amount']), bins=100, kde = True, color = 'green')  
plt.title('Transaction Amount Distribution(log scale)')  
plt.xlabel('log(Amount + 1)')  
plt.show()
```



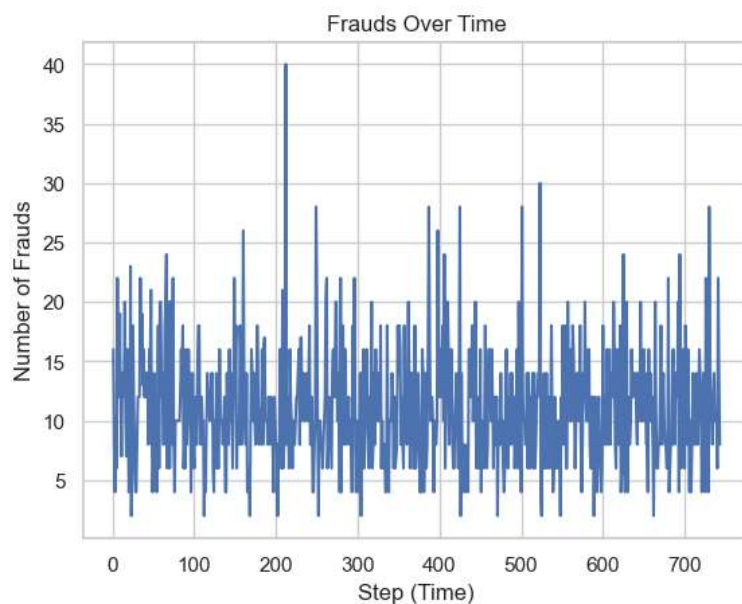
- Analyzed the relationship between fraudulent transactions and transaction amounts to determine how transaction value influences fraud occurrence.

```
sns.boxplot(data= df[df['amount'] < 50000], x = 'isFraud', y = 'amount')
plt.title('Amount vs isFraud (Filter under 50k)')
plt.show()
```



- Analyzed fraud trends over time to determine whether transaction timing influenced fraudulent activity.
- The analysis showed that fraud occurrence was largely time-independent; therefore, the time-related variable was excluded from the machine learning model to improve model simplicity and efficiency.

```
frauds_per_step = df[df['isFraud'] == 1]['step'].value_counts().sort_index()
plt.plot(frauds_per_step.index, frauds_per_step.values, label = 'Frauds per Step')
plt.xlabel('Step (Time)')
plt.ylabel('Number of Frauds')
plt.title('Frauds Over Time')
plt.grid(True)
plt.show()
```



```
df.drop(columns = 'step', inplace = True)
```

```
df.head()
```

|   | type     | amount   | nameOrig    | oldbalanceOrig | newbalanceOrig | nameDest    | oldbalanceDest | newbalanceDest | isFraud | isFlaggedFraud | balanceDiffOrig | balanceDest |
|---|----------|----------|-------------|----------------|----------------|-------------|----------------|----------------|---------|----------------|-----------------|-------------|
| 0 | PAYMENT  | 9839.64  | C1231006815 | 170136.0       | 160296.36      | M1979787155 | 0.0            | 0.0            | 0       | 0              | 9839.64         |             |
| 1 | PAYMENT  | 1864.28  | C1666544295 | 21249.0        | 19384.72       | M2044282225 | 0.0            | 0.0            | 0       | 0              | 1864.28         |             |
| 2 | TRANSFER | 181.00   | C1305486145 | 181.0          | 0.00           | C553264065  | 0.0            | 0.0            | 1       | 0              | 181.00          |             |
| 3 | CASH_OUT | 181.00   | C840083671  | 181.0          | 0.00           | C38997010   | 21182.0        | 0.0            | 1       | 0              | 181.00          |             |
| 4 | PAYMENT  | 11668.14 | C2048537720 | 41554.0        | 29885.86       | M1230701703 | 0.0            | 0.0            | 0       | 0              | 11668.14        |             |

- Identified and analyzed customers responsible for the highest transaction amounts for both senders and receivers.
- The analysis helped detect unusual transaction behavior and potential high-risk accounts associated with large financial transfers.

```
#Customers which make the highest transactions
```

```
top_senders = df["nameOrig"].value_counts().head(10)
top_senders
```

```
nameOrig
C1902386530    3
C363736674     3
C545315117     3
C724452879     3
C1784010646    3
C1677795071    3
C1462946854    3
C1999539787    3
C2098525306    3
C400299098     3
Name: count, dtype: int64
```

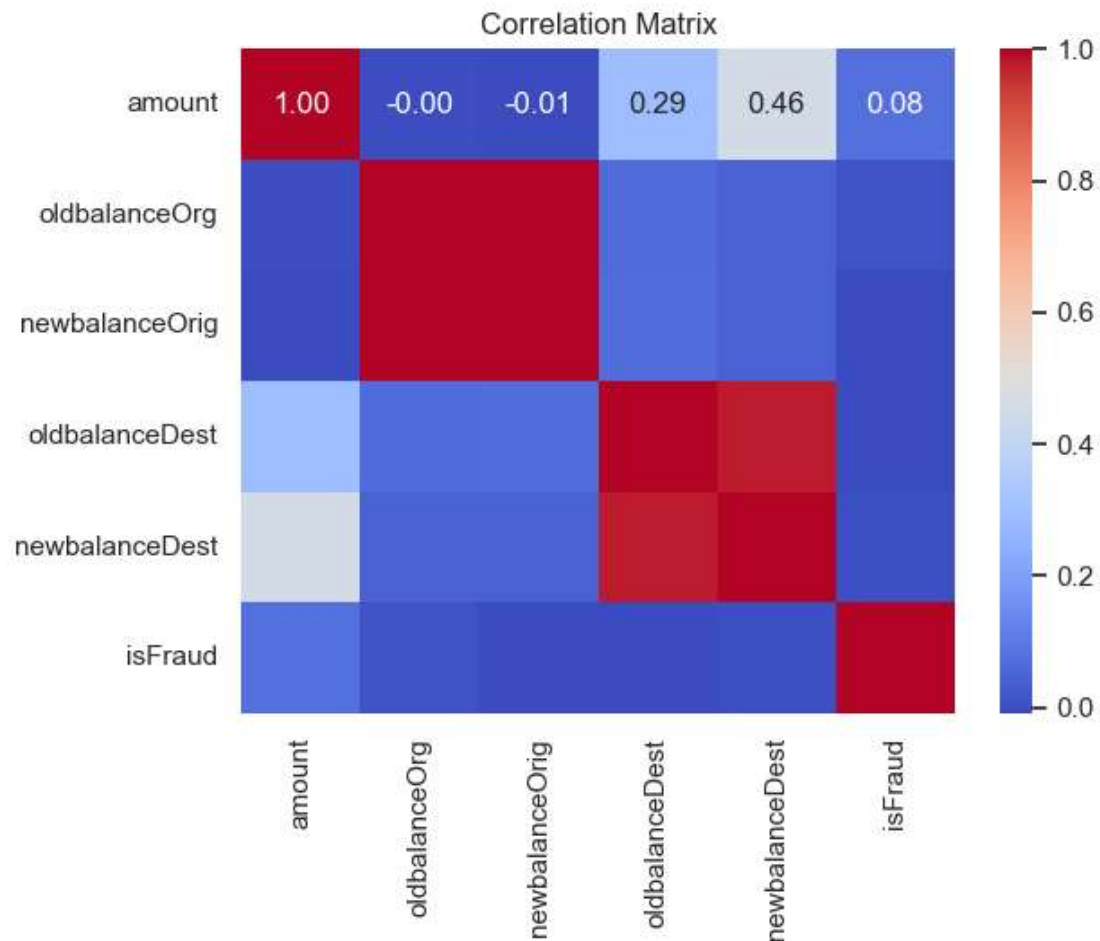
```
top_recievers = df['nameDest'].value_counts().head(10)
top_recievers
```

```
nameDest
C1286084959    113
C985934102     109
C665576141     105
C2083562754    102
C248609774     101
C1590550415    101
C451111351     99
C1789550256    99
C1360767589    98
C1023714065    97
Name: count, dtype: int64
```



- Generated a correlation matrix using a heat map to analyze the relationships between different variables in the dataset.
- The visualization helped identify strong positive and negative correlations between features, supporting feature selection and machine learning model development.

```
sns.heatmap(corr, annot = True, cmap = 'coolwarm', fmt = '.2f')  
plt.title('Correlation Matrix')  
plt.show()
```



- Imported machine learning tools and libraries required for data preprocessing, model training, evaluation, and pipeline development.
- Dropped unnecessary columns and irrelevant variables to improve model performance, reduce complexity, and ensure the model was trained using only relevant features.

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

df.head()

```

|   | type     | amount   | nameOrig    | oldbalanceOrig | newbalanceOrig | nameDest    | oldbalanceDest | newbalanceDest | isFraud | isFlaggedFraud | balanceDiffOrig | balance |
|---|----------|----------|-------------|----------------|----------------|-------------|----------------|----------------|---------|----------------|-----------------|---------|
| 0 | PAYMENT  | 9839.64  | C1231006815 | 170136.0       | 160296.36      | M1979787155 | 0.0            | 0.0            | 0       | 0              | 9839.64         |         |
| 1 | PAYMENT  | 1864.28  | C1666544295 | 21249.0        | 19384.72       | M2044282225 | 0.0            | 0.0            | 0       | 0              | 1864.28         |         |
| 2 | TRANSFER | 181.00   | C1305486145 | 181.0          | 0.00           | C553264065  | 0.0            | 0.0            | 1       | 0              | 181.00          |         |
| 3 | CASH_OUT | 181.00   | C840083671  | 181.0          | 0.00           | C38997010   | 21182.0        | 0.0            | 1       | 0              | 181.00          |         |
| 4 | PAYMENT  | 11668.14 | C2048537720 | 41554.0        | 29885.86       | M1230701703 | 0.0            | 0.0            | 0       | 0              | 11668.14        |         |

```

df_model = df.drop(['nameOrig', 'nameDest', 'isFlaggedFraud'], axis = 1)

```

- Defined the target variable and feature variables required for fraud prediction.
- Set up the machine learning pipeline to automate data preprocessing and model training processes.
- Trained and tested the model using training and testing datasets to evaluate prediction accuracy and overall model performance.

```
categorical = ['type']
numeric = ['amount', 'oldbalanceOrig', 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest']
```

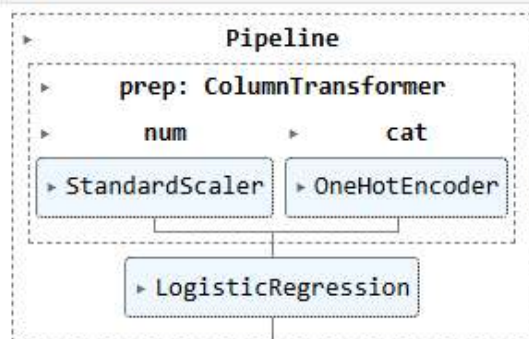
```
y = df_model['isFraud']
X = df_model.drop('isFraud', axis = 1)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, stratify = y)
```

```
preprocessor = ColumnTransformer(
    transformers = [
        ('num', StandardScaler(), numeric),
        ('cat', OneHotEncoder(drop='first'), categorical)
    ],
    remainder = 'drop'
)
```

```
pipeline = Pipeline([
    ('prep', preprocessor),
    ('clf', LogisticRegression(class_weight = 'balanced', max_iter = 1000))
])
```

```
pipeline.fit(X_train, y_train)
```



```
y_pred = pipeline.predict(X_test)
y_pred
```

```
array([0, 0, 0, ..., 0, 0, 0], dtype=int64)
```

- Evaluated alternative machine learning methods and compared their performance to identify the most effective fraud detection model.
- The final selected model achieved a precision score of 94.78%, indicating strong predictive performance and adequate reliability in detecting fraudulent transactions.

```

103
: print(classification_report(y_test,y_pred))

              precision    recall  f1-score   support

     0           1.00        0.95        0.97    1906322
     1           0.02        0.93        0.04       2464

 accuracy          0.95    1908786
 macro avg          0.51    1908786
weighted avg          1.00    1908786

: confusion_matrix(y_test,y_pred)

: array([[1806816,  99506],
        [   161,   2303]], dtype=int64)

: #Precision Percentage
: pipeline.score(X_test, y_test) * 100

: 94.77851367308855

```

- Saved the trained machine learning pipeline for future fraud prediction and deployment within the Streamlit application.
- This enabled the model to be reused efficiently without retraining during real-time transaction prediction processes.

```

import joblib
joblib.dump(pipeline, 'fraud_detection_pipeline.pkl')

['fraud_detection_pipeline.pkl']

```

# Fraud Detection Prediction Application

Streamlit-based web application that loads a trained machine learning model and predicts whether a financial transaction is fraudulent based on user-input transaction details.

```
1 import streamlit as st
2 import pandas as pd
3 import joblib
4
5 model = joblib.load(r'C:\Users\Corquip\Downloads\fraud_detection_pipeline.pkl')
6
7 st.title('Fraud Detection Prediction App')
8
9 st.markdown('Please enter the transaction details and use the predict button')
10
11 st.divider()
12
13 transaction_type = st.selectbox('Transaction Type', ['PAYMENT', 'TRANSFER', 'CASH_OUT', 'DEPOSIT'])
14 amount = st.number_input('Amount', min_value = 0.0, value = 1000.00)
15 oldbalanceOrg = st.number_input('Old Balance (Sender)', min_value = 0.0, value = 10000.00)
16 newbalanceOrig = st.number_input('New Balance (Sender)', min_value = 0.0, value = 9000.00)
17 oldbalanceDest = st.number_input('Old Balance (Reciever)', min_value = 0.0, value = 0.0)
18 newbalanceDest = st.number_input('New Balance (Reciever)', min_value = 0.0, value = 0.0)
19
20 if st.button('Predict'):
21     input_data = pd.DataFrame([
22         {
23             'type' : transaction_type,
24             'amount' : amount,
25             'oldbalanceOrg' : oldbalanceOrg,
26             'newbalanceOrig' : newbalanceOrig,
27             'oldbalanceDest' : oldbalanceDest,
28             'newbalanceDest' : newbalanceDest
29         }
30     ])
31
32     prediction = model.predict(input_data)[0]
33
34     st.subheader(f'Prediction : '{int(prediction)}')
35
36     if prediction == 1:
37         st.error('This Transaction can be fraud')
38
39     else:
40         st.success('This transaction looks like it is not a fraud')
```

# FRAUD DETECTION APP

Deploy

## Fraud Detection Prediction App

Please enter the transaction details and use the predict button

---

Transaction Type

CASH\_OUT

Amount

1000.00

Old Balance (Sender)

100000000.00

New Balance (Sender)

0.00

Old Balance (Reciever)

0.00

New Balance (Reciever)

0.00

Predict

Prediction : '1'

This Transaction can be fraud

## CONCLUSION

The Fraud Detection Prediction Application successfully demonstrated the effectiveness of Machine Learning in identifying fraudulent financial transactions. By integrating data preprocessing, predictive modeling, and a user-friendly Streamlit interface, the system was able to provide real-time fraud predictions based on transaction details. The application achieved a precision score of 94%, indicating strong predictive performance and reliability in detecting fraudulent activities while minimizing incorrect fraud alerts. Overall, the project highlights how machine learning can support financial institutions in improving transaction security, reducing financial losses, and enhancing fraud prevention strategies.